

claude-windows / 9b717b4b-673e-43c5-b3f4-33099e782c45

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9b717b4b-673e-43c5-b3f4-33099e782c45\subagents\workflows\wf_711ef326-a91\agent-af584d38980e7f3ea.jsonl`  
- SHA-256: `a2110e5f9e473277723ba6de2f1c32171fc07c8dfa869b4d760635e90b686bbc`  
- Source modified: `2026-06-08T11:35:35+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_711ef326-a91`  
- session_id: `9b717b4b-673e-43c5-b3f4-33099e782c45`
```

Transcript

1. user (2026-06-08T11:34:33.026Z)

Read C:\Users\avidu\Projects\rally-annotator\vlc\rally_annotator.lua in full. Review ONLY the LOGIC/edge-cases of the new "Next rally #" resume feature and its interaction with existing flows. planned_next_number() reads w_next (floors values >=1, else falls back to next_rally_number()). save_rally() new-rally branch uses it, refuses if index_of_rally(n) exists, appends, then sets w_next to n+1. refresh_next_field() resets w_next to next_rally_number() and is called on activate/create_dialog and in sync_to_current_video. Check edge cases: does EDIT mode wrongly use planned_next_number (it must use the existing rally number, not w_next)? What if w_next is blank, 0, negative, or fractional? Could the collision check be bypassed? After a refused duplicate, is state consistent (no row written, w_next unchanged)? Does delete/undo interact badly with a user-set w_next (e.g., stale value causing surprising next number)? Is the auto-advance correct when inserting into a gap? Does the Save button label (planned_next_number) stay consistent with what save actually writes? Report concrete defects only.

2. assistant (2026-06-08T11:34:34.827Z)

I'll read the file in full and analyze the resume feature logic.

3. user (2026-06-08T11:34:36.052Z)

```
1      --[[ rally_annotator.lua  --  VLC Lua EXTENSION (v1.4)
2
3      Rally Annotator for NET-SEPARATED RACQUET SPORTS
4      (badminton · tennis · table tennis · pickleball · padel)
5
6      While watching a match in VLC, mark each rally's START / END and the point-stop
7      reason, and append one CSV row per rally. Pause/scrub freely, then click ? the
8      callback snapshots the exact playback time. Output is a plain CSV that ingests
9      directly into common rally-segmentation tooling.
10
11      Output CSV columns (times in decimal SECONDS):
12          rally_number,start_time,end_time,ending_reason,sport
13
14      WHAT'S NEW IN v1.3
15      - In-dialog HELP button: usage + an ending-reason decision guide, rendered
16        inside the extension's own dialog (see "WHERE HELP LIVES" below).
17      - Two-step commit: Mark END now ARMS a rally; you pick the (required) reason
18        and click "Save Rally" to write it. The reason is RESET after every save,
19        so it can never silently reuse the previous rally's reason.
20      - Editable Start/End fields (fine-tune the marked seconds before saving).
21      - "Recent rallies" list with Edit selected / Delete selected, plus Undo last.
22
23      WHERE HELP LIVES (VLC internals, verified against the 3.0.x source):
24      The empty "About <name>.lua" window under Tools > Plugins and extensions >
```

```

25     Add-ons is VLC's AddonInfoDialog; its body text ("Lua script") is a hardcoded
26     constant supplied by VLC's filesystem addon scanner (modules/misc/addons/
27     fsstorage.c) and CANNOT be set by an extension. The descriptor() shortdesc/
28     description below DO show, but in a different place ? the "More information"
29     dialog reached from the *Active Extensions* tab. For real, author-controlled
30     help we therefore render it INSIDE this extension's dialog (the HELP button).
31
32     WHY a button-dialog (not an auto-pause hook):
33     VLC 3.x can expose pause via the {"playing-listener"} capability
34     (status_changed/playing_changed), but that callback is flaky on macOS
35     (VLC #22778) and the input/meta listeners are broken in VLC 4.0 (#27558).
36     A persistent dialog whose button callbacks SNAPSHOT the current playback
37     time is portable, precise, and lets the rater pause/scrub before committing.
38     So we declare NO listener capabilities.
39
40     UNITS: in VLC 3.x, vlc.var.get(input,"time") is in MICROSECONDS -> /1e6.
41
42     Widget grid args are (col, row, colspan, rowspan), 1-indexed.
43
44     Install (Windows): copy this file to
45         %APPDATA%\vlc\lua\extensions\rally_annotator.lua
46     macOS: ~/Library/Application Support/org.videolan.vlc/lua/extensions/
47     Linux: ~/.local/share/vlc/lua/extensions/
48     then VLC > Tools > Plugins and extensions > Reload extensions (or restart),
49     then enable it from the View menu.
50
51     MIT licensed. https://github.com/avidullu/rally-annotator
52 ]]
53
54 -----
55 -- Extension registration
56 -----
57 function descriptor()
58     return {
59         title      = "Rally Annotator",
60         version    = "1.4",
61         author     = "Avi Dullu",
62         url        = "https://github.com/avidullu/rally-annotator",
63         shortdesc  = "Mark rally start/end + a point-ending reason to a CSV (net-separated
racquet sports)",
64         description =
65             "Mark each rally's START and END while you watch, tag WHY the point ended "
66             .. "(winner / forced_error / unforced_error / service_fault / let / other), and
append "
67             .. "one CSV row per rally next to the video "
68             .. "(rally_number,start_time,end_time,ending_reason,sport; decimal seconds). "
69             .. "Two-step Save (Mark END, then Save Rally) with a REQUIRED, non-sticky reason; "
70             .. "editable times; edit or delete recent rallies. "
71             .. "For badminton/tennis/table-tennis/pickleball/padel. "
72             .. "Click the HELP button inside the dialog for usage + an ending-reason decision
guide.",
73         capabilities = {}    -- button-click model: no listeners needed
74     }
75 end
76
77 -----
78 -- Config / constants
79 -----
80 -- Net-separated racquet sports share a forced/unforced-error point-stop taxonomy.
81 local REASON_PLACEHOLDER = "-- choose reason --"
82 local REASONS = {
83     "winner", "forced_error", "unforced_error", "service_fault", "let", "other"
84 }
85 local SPORTS = {
86     "badminton", "tennis", "table_tennis", "pickleball", "padel"

```

```

87     }
88     local HEADER = "rally_number,start_time,end_time,ending_reason,sport\n"
89
90     -- reason/sport -> id lookups. POPULATED IN activate(), NOT here: VLC scans an
91     -- extension's descriptor() in a restricted Lua sandbox that lacks base globals
92     -- like ipairs, so calling ipairs at top level makes the whole extension fail to
93     -- load (it then never appears under the View menu).
94     local REASON_ID = {}
95     local SPORT_ID = {}
96
97     -- In-dialog help (rendered into the status panel by the HELP button). Basic HTML
98     -- only (<b>, <i>, <br>) for portability across the Qt and macOS dialog renderers.
99     local HELP_HTML = [=[
100     <b>Rally Annotator &mdash; how to use</b><br>
101     1. Pick the <b>Sport</b> (top). It stays set across rallies.<br>
102     2. When a rally begins, click <b>Mark START</b> (pause/scrub first for frame accuracy
&mdash; it snapshots the current playback time into the Start field).<br>
103     3. When the rally ends, click <b>Mark END</b>. You may fine-tune the Start/End seconds
by editing those fields directly.<br>
104     4. Choose the <b>Ending reason</b> (REQUIRED) and click <b>Save Rally</b> &mdash; one
CSV row is written next to the video.<br>
105     5. The reason RESETS after every save, so it is never reused by accident.<br>
106     6. <b>Recent rallies</b>: select a row, then <b>Edit selected</b> (loads it back into
the fields) or <b>Delete selected</b>. <b>Undo last</b> removes the most recent row (the button
shows which, e.g. <i>Undo last (#7)</i>), or clears an in-progress mark, or cancels an edit.<br>
107     7. <b>Resuming later:</b> labels are saved to the CSV next to the video as you go. Re-
open the SAME video and enable the extension &mdash; it reloads your existing rallies and
continues numbering. Set the <b>Next rally #</b> field (top right) to resume numbering from any
value (e.g. restart at 1, or continue from 50); it auto-advances after each save. If you switch
videos with this dialog open, click <b>Refresh</b> to load the current video's rallies. (The
video's playback position is not restored &mdash; scrub to where you stopped.)<br>
108     <br>
109     <b>Ending reasons &mdash; what they mean (pick the one that says WHY the rally
ended).</b><br>
110     All reasons except <i>winner</i> are charged to the side that LOST the rally.<br>
111     &bull; <b>winner</b> &mdash; the last shot landed IN and was not returned (opponent
couldn't reach it, or only waved at it). A clean untouched serve (ace) counts here.<br>
112     &bull; <b>forced_error</b> &mdash; the loser MISSED (out, or into the net) while UNDER
PRESSURE: stretched, rushed, jammed, or handling the opponent's pace/spin/depth. They were made
to miss.<br>
113     &bull; <b>unforced_error</b> &mdash; the loser MISSED a routine shot they had time AND
position to make, with little or no pressure. They gave it away.<br>
114     &bull; <b>service_fault</b> &mdash; the point ended on the SERVE: serve into the net,
out of the service box, illegal action/foot fault, or a double fault (tennis/padel).<br>
115     &bull; <b>let</b> &mdash; the rally is REPLAYED under the rules (no point): e.g. a
tennis/table-tennis serve that clips the net and is otherwise good, or outside interference.<br>
116     &bull; <b>other</b> &mdash; anything else (occluded footage, injury/retirement, penalty,
hindrance). Use sparingly.<br>
117     <br>
118     <b>Quick cases</b><br>
119     &bull; Ball/shuttle lands <b>OUT</b> (past the baseline / outside the lines): it is the
hitter's miss &mdash; <b>forced</b> if they were under pressure, <b>unforced</b> if it was a
comfortable ball. (OUT is NOT automatically forced, and never a winner for the opponent.)<br>
120     &bull; <b>Into the net</b> during a rally: same &mdash; forced if pressured, unforced if
routine. A <b>serve</b> into the net is <b>service_fault</b>.<br>
121     &bull; <b>Net-cord that dribbles over and lands in</b>: live and good &mdash; usually a
<b>winner</b> if unreachable, else judge what happens next. On the SERVE it varies by sport:
tennis/table-tennis = <b>let</b> (replay); badminton net-tick that passes over and lands in =
play on (but a serve shuttle CAUGHT on the net = service_fault); pickleball (2026) = play
on.<br>
122     &bull; <b>Unsure forced vs unforced?</b> Default to <b>unforced</b> &mdash; only mark
forced when you can point to the specific pressure.<br>
123     <br>
124     Output: <i>&lt;video>.rallies.csv</i> next to the video. Full guide:
docs/ENDING_REASONS.md in the repo.<br>

```

```

125 Click <b>Hide help</b> to return to the status panel.
126 ]==]
127
128 -----
129 -- State
130 -----
131 local d                -- the single dialog (one per extension)
132 local w_status         -- status label widget (HTML/rich-text); also hosts help
133 local w_reason         -- ending_reason dropdown widget
134 local w_sport          -- sport dropdown widget
135 local w_start          -- start-time text input (editable seconds)
136 local w_end            -- end-time text input (editable seconds)
137 local w_list           -- recent-rallies list widget
138 local w_save           -- "Save Rally" / "Save changes" button (reabeled by state)
139 local w_undo           -- "Undo last" button (reabeled to show which row it removes)
140 local w_next           -- "Next rally #" text input (lets you resume numbering
anywhere)
141 local w_help_btn       -- the Help button (reabeled "Hide help" when open)
142 local w_help           -- the help panel widget; nil when hidden (toggled via
del_widget)
143
144 local rows = {}        -- in-memory rally rows: { n, s, e, reason, sport, [extra] }
145 local mode = "new"     -- "new" (marking a fresh rally) or "edit" (editing a row)
146 local edit_index = nil -- index into rows when mode == "edit"
147 local out_path         -- resolved CSV path (set on activate; re-resolved by Refresh)
148
149 -----
150 -- Helpers (declared before the global callbacks that capture them)
151 -----
152
153 -- Current playback time in SECONDS, or nil if nothing is playing.
154 local function now_seconds()
155     local input = vlc.object.input()
156     if not input then return nil end
157     local t_us = vlc.var.get(input, "time") -- MICROSECONDS in VLC 3.x
158     if not t_us then return nil end
159     return t_us / 1000000.0
160 end
161
162 -- mm:ss.mmm for display only.
163 local function fmt_clock(s)
164     if not s then return "--:--" end
165     if s < 0 then s = 0 end
166     local m = math.floor(s / 60)
167     local sec = s - m * 60
168     return string.format("%d:%06.3f", m, sec)
169 end
170
171 -- Minimal HTML-escape so paths/messages render literally in the rich-text label.
172 local function esc(text)
173     text = tostring(text or "")
174     text = text:gsub("&", "&")
175     text = text:gsub("<", "<")
176     text = text:gsub(">", ">")
177     return text
178 end
179
180 -- Best-effort path to the currently playing media file (decoded from URI).
181 local function current_media_path()
182     local input = vlc.object.input()
183     if not input then return nil end
184     local item = vlc.input.item() -- VLC 3.x: vlc.input.item()
185     if not item then return nil end
186     local uri = item.uri() -- item.uri()
187     if not uri or uri == "" then return nil end

```

```

188     local p = uri
189     -- Strip the scheme. Windows file URIs look like file:///C:/dir/clip.mp4;
190     -- POSIX ones like file:///home/user/clip.mp4 -- keep the POSIX leading "/".
191     if p:match("^file:///a:") then
192         p = p:gsub("^file://", "") -- file:///C:/... -> C:/...
193     else
194         p = p:gsub("^file://", "") -- file:///home/... -> /home/...
195     end
196     -- percent-decode (e.g. %20 -> space) BEFORE separator normalization
197     p = p:gsub("%%(%x%x)", function(h) return string.char(tonumber(h, 16)) end)
198     if p:match("^a:") then -- Windows drive path -> backslashes
199         p = p:gsub("/", "\\")
200     end
201     return p
202 end
203
204 -- Resolve a sensible default output path: next to the video if we can, else
205 -- the user's home (USERPROFILE on Windows, HOME elsewhere).
206 local function resolve_out_path()
207     local media = current_media_path()
208     if media then
209         local dir, stem = media:match("^([^\\\/])([\\\/-])%.?([\\\/%.]*)$")
210         if dir and stem and stem ~= "" then
211             return dir .. stem .. ".rallies.csv"
212         end
213         local d2 = media:match("^([^\\\/])")
214         if d2 then return d2 .. "rally_labels.csv" end
215     end
216     local home = os.getenv("USERPROFILE") or os.getenv("HOME") or "."
217     local sep = home:find("\\") and "\\" or "/"
218     if home:sub(-1) ~= sep then home = home .. sep end
219     return home .. "rally_labels.csv"
220 end
221
222 -- Load all rally rows from the CSV into `rows` (header + blank lines skipped).
223 -- Forgiving parser: any leading-integer line is a rally row; extra columns are
224 -- preserved verbatim so we never drop richer metadata on rewrite.
225 local function load_rows()
226     rows = {}
227     local f = io.open(out_path, "r")
228     if not f then return end
229     for line in f:lines() do
230         line = line:gsub("\r$", "")
231         if line:match("%S") then
232             local parts = {}
233             for field in (line .. ","):gmatch("([^\,]*)",) do parts[#parts + 1] = field end
234             local n = tonumber(parts[1])
235             if n and n == math.floor(n) then
236                 local row = {
237                     n = math.floor(n),
238                     s = tonumber(parts[2]) or 0,
239                     e = tonumber(parts[3]) or 0,
240                     reason = (parts[4] ~= nil and parts[4] ~= "") and parts[4] or "other",
241                     sport = parts[5] or "",
242                 }
243                 if #parts > 5 then
244                     local ex = {}
245                     for i = 6, #parts do ex[#ex + 1] = parts[i] end
246                     row.extra = table.concat(ex, ",")
247                 end
248                 rows[#rows + 1] = row
249             end
250         end
251     end
252     f:close()

```

```

253 end
254
255 -- Rewrite the whole CSV from `rows` (header once). We write a temp file and swap
256 -- it into place so the live CSV is never lost if a step fails: rename is atomic on
257 -- POSIX (and succeeds for a brand-new file on Windows); when Windows can't rename
258 -- over an existing file we stash the original as .bak and roll back on error.
259 local function save_all()
260     local tmp = out_path .. ".tmp"
261     local f, err = io.open(tmp, "w")
262     if not f then return false, ("cannot open CSV for write: " .. tostring(err)) end
263     f:write(HEADER)
264     for _, r in ipairs(rows) do
265         local line = string.format("%d,%.3f,%.3f,%s,%s", r.n, r.s, r.e, r.reason, r.sport)
266         if r.extra and r.extra ~= "" then line = line .. "," .. r.extra end
267         f:write(line .. "\n")
268     end
269     f:close()
270     -- Atomic on POSIX; also succeeds on Windows when out_path doesn't exist yet.
271     if os.rename(tmp, out_path) then return true end
272     -- Windows overwrite path: keep a backup so the original is never lost.
273     local bak = out_path .. ".bak"
274     os.remove(bak)
275     local stashed = os.rename(out_path, bak) -- move original aside (nil if none)
276     local ok, rerr = os.rename(tmp, out_path)
277     if not ok then
278         if stashed then os.rename(bak, out_path) end -- roll back
279         return false, ("cannot replace CSV: " .. tostring(rerr))
280     end
281     os.remove(bak)
282     return true
283 end
284
285 -- Next monotonic rally_number (max existing + 1); keeps numbering across reopen.
286 local function next_rally_number()
287     local maxn = 0
288     for _, r in ipairs(rows) do if r.n > maxn then maxn = r.n end end
289     return maxn + 1
290 end
291
292 -- Read a text-input widget as a number, or nil if blank/non-numeric.
293 local function get_field_num(w)
294     if not w then return nil end
295     local t = w:get_text()
296     if not t then return nil end
297     t = t:gsub("%s+", "")
298     if t == "" then return nil end
299     return tonumber(t)
300 end
301
302 -- The number the NEXT new rally will get. Defaults to next_rally_number(), but the
303 -- user can type any value into the "Next rally #" field to resume/insert anywhere.
304 local function planned_next_number()
305     local v = get_field_num(w_next)
306     if v and v >= 1 then return math.floor(v) end
307     return next_rally_number()
308 end
309
310 -- Reset the "Next rally #" field to the natural next number for the current CSV.
311 local function refresh_next_field()
312     if w_next then w_next:set_text(tostring(next_rally_number())) end
313 end
314
315 local function index_of_rally(n)
316     for i, r in ipairs(rows) do if r.n == n then return i end end
317     return nil

```

```

318     end
319
320     -- First selected rally_number in the recent list, or nil. get_selection() returns
321     -- a { [id]=text } table in VLC 3.0.x (id is the rally_number we stored).
322     local function selected_rally_number()
323         if not w_list then return nil end
324         local sel = w_list:get_selection()
325         if type(sel) ~= "table" then return nil end
326         for id, _ in pairs(sel) do
327             return tonumber(id) or id
328         end
329         return nil
330     end
331
332     -- Dropdowns have no set_value in 3.0.x; the FIRST add_value after a clear() is
333     -- auto-selected. So we reset/select by clear()+rebuild with the desired value first.
334     local function rebuild_reason_placeholder()
335         if not w_reason then return end
336         w_reason:clear()
337         w_reason:add_value(REASON_PLACEHOLDER, 0)          -- id 0 = "not chosen"
338         for i, v in ipairs(REASONS) do w_reason:add_value(v, i) end
339     end
340
341     local function rebuild_reason_selected(sel)
342         if not w_reason then return end
343         w_reason:clear()
344         local id = REASON_ID[sel] or 99
345         w_reason:add_value(sel, id)                        -- first => shown selected
346         for i, v in ipairs(REASONS) do
347             if v ~= sel then w_reason:add_value(v, i) end
348         end
349     end
350
351     local function rebuild_sport_selected(sel)
352         if not w_sport then return end
353         if not sel or sel == "" then sel = SPORTS[1] end
354         w_sport:clear()
355         local id = SPORT_ID[sel] or 99
356         w_sport:add_value(sel, id)                         -- first => shown selected
357         for i, v in ipairs(SPORTS) do
358             if v ~= sel then w_sport:add_value(v, i) end
359         end
360     end
361
362     -- Repaint the recent-rallies list (last 12 rows).
363     local function refresh_list()
364         if not w_list then return end
365         w_list:clear()
366         local total = #rows
367         local starti = total - 11
368         if starti < 1 then starti = 1 end
369         for i = starti, total do
370             local r = rows[i]
371             w_list:add_value(
372                 string.format("#%d    %s -> %s    [%s, %s]", r.n, fmt_clock(r.s), fmt_clock(r.e),
373                 r.reason, r.sport),
374             end
375             if d then d:update() end
376         end
377
378         -- Action-button labels reflect state. Save shows the rally number it will write
379         -- (or "Save changes (#N)" while editing). Undo last shows exactly which row it
380         -- will remove ("Undo last (#N)"), or that it will cancel an edit / clear a mark.
381         local function refresh_buttons()

```

```

382     if w_save then
383         if mode == "edit" and edit_index and rows[edit_index] then
384             w_save:set_text(string.format("Save changes (##d)", rows[edit_index].n))
385         else
386             local s = get_field_num(w_start)
387             local e = get_field_num(w_end)
388             if s and e then
389                 w_save:set_text(string.format("Save Rally (##d)", planned_next_number()))
390             else
391                 w_save:set_text("Save Rally")
392             end
393         end
394     end
395     if w_undo then
396         if mode == "edit" then
397             w_undo:set_text("Undo last (cancel edit)")
398         else
399             local s = get_field_num(w_start)
400             local e = get_field_num(w_end)
401             if s or e then
402                 w_undo:set_text("Undo last (clear mark)")
403             elseif #rows > 0 then
404                 w_undo:set_text(string.format("Undo last (##d)", rows[#rows].n))
405             else
406                 w_undo:set_text("Undo last")
407             end
408         end
409     end
410     if d then d:update() end
411 end
412
413 local function set_status(msg)
414     if not w_status then return end
415     local mode_line
416     if mode == "edit" and edit_index and rows[edit_index] then
417         mode_line = string.format(
418             "Mode: EDITING rally ##d (Save changes to commit, Undo last to cancel).",
419             rows[edit_index].n)
420     else
421         mode_line = "Mode: new rally (Mark START, Mark END, choose reason, Save Rally)."
422     end
423     local last_line = ""
424     if #rows > 0 then
425         local lr = rows[#rows]
426         last_line = string.format("Last row (what Undo removes): ##d %s -> %s",
427             lr.n, esc(fmt_clock(lr.s)), esc(fmt_clock(lr.e)), esc(lr.reason))
428     end
429     w_status:set_text(string.format(
430         "%s<br>%s<br>%sNow: %s &nbsp;|&nbsp; Rallies in CSV: %d<br>CSV: %s",
431         esc(msg), esc(mode_line), last_line, esc(fmt_clock(now_seconds())), #rows,
432         esc(out_path)))
433     if d then d:update() end
434 end
435
436 -- Clear the form back to a fresh "new rally" state (reason reset = non-sticky).
437 local function reset_form()
438     mode = "new"
439     edit_index = nil
440     if w_start then w_start:set_text("") end
441     if w_end then w_end:set_text("") end
442     rebuild_reason_placeholder()
443     refresh_buttons()
444     if d then d:update() end
445 end

```



```

444
445 -- Re-resolve the CSV path against whatever video is playing NOW and load its
446 -- labels. Lets the user open a video later (or switch videos) and pick up exactly
447 -- where they left off without toggling the extension off/on. Same video -> just
448 -- re-read the CSV; a different video -> repoint and load that video's rallies.
449 local function sync_to_current_video()
450     if mode == "edit" then
451         -- Reloading rows would invalidate edit_index; make the user resolve the edit first.
452         refresh_list(); refresh_buttons()
453         set_status("Finish (Save changes) or cancel (Undo last) the current edit before
refreshing.")
454         return
455     end
456     local media = current_media_path()
457     if not media then
458         refresh_list(); refresh_buttons()
459         set_status("No media is playing -- still using this CSV. Open the video, then click
Refresh.")
460         return
461     end
462     local new_path = resolve_out_path()
463     if new_path == out_path then
464         load_rows() -- re-read in case the file changed on disk
465         refresh_next_field()
466         refresh_list(); refresh_buttons()
467         set_status(string.format("Refreshed. %d rallies loaded for this video.", #rows))
468         return
469     end
470     out_path = new_path -- switched videos -> repoint + resume that file
471     load_rows()
472     reset_form()
473     refresh_next_field()
474     refresh_list()
475     refresh_buttons()
476     set_status(string.format(
477         "Switched to the current video. Loaded %d existing rallies; next is #%d.", #rows,
next_rally_number()))
478     end
479
480 -----
481 -- Button callbacks (VLC calls these on its main loop; kept global to be safe)
482 -----
483 function mark_start()
484     local t = now_seconds()
485     if not t then set_status("No media playing -- cannot mark START."); return end
486     w_start:set_text(string.format("%.3f", t))
487     refresh_buttons()
488     set_status(string.format("START set @ %s. Play to the rally's end, then Mark END.",
fmt_clock(t)))
489     end
490
491 function mark_end()
492     local t = now_seconds()
493     if not t then set_status("No media playing -- cannot mark END."); return end
494     w_end:set_text(string.format("%.3f", t))
495     refresh_buttons()
496     set_status(string.format("END set @ %s. Choose the Ending reason, then click Save
Rally.", fmt_clock(t)))
497     end
498
499 function save_rally()
500     local s = get_field_num(w_start)
501     local e = get_field_num(w_end)
502     if not s then set_status("Set a START time first (click Mark START)."); return end
503     if not e then set_status("Set an END time first (click Mark END)."); return end

```

```

504     if e < s then s, e = e, s end    -- tolerate reversed marks
505     if e <= s then
506         set_status("END must be later than START (rally must be > 0s).")
507         return
508     end
509     local rid, reason = w_reason:get_value()    -- (id, text)
510     if not rid or rid == 0 or not reason or reason == "" or reason == REASON_PLACEHOLDER
then
511         set_status("Choose an Ending reason -- it is required (still on \" " ..
REASON_PLACEHOLDER .. "\").")
512         return
513     end
514     local _, sport = w_sport:get_value()
515     if not sport or sport == "" then sport = SPORTS[1] end
516
517     local msg
518     if mode == "edit" and edit_index and rows[edit_index] then
519         local r = rows[edit_index]
520         r.s, r.e, r.reason, r.sport = s, e, reason, sport
521         local ok, err = save_all()
522         if not ok then set_status("WRITE FAILED: " .. tostring(err)); return end
523         msg = string.format("Updated rally #%d: %s -> %s [%s, %s].", r.n, fmt_clock(s),
fmt_clock(e), reason, sport)
524     else
525         local n = planned_next_number()
526         if index_of_rally(n) then
527             set_status(string.format("Rally #%d already exists -- set \"Next rally #\" to a
free number.", n))
528             return
529         end
530         rows[#rows + 1] = { n = n, s = s, e = e, reason = reason, sport = sport }
531         local ok, err = save_all()
532         if not ok then rows[#rows] = nil; set_status("WRITE FAILED: " .. tostring(err));
return end
533         if w_next then w_next:set_text(tostring(n + 1)) end    -- continue numbering from
here
534         msg = string.format("Saved rally #%d: %s -> %s [%s, %s].", n, fmt_clock(s),
fmt_clock(e), reason, sport)
535     end
536
537     reset_form()    -- clears fields + resets the (required) reason to placeholder
538     refresh_list()
539     set_status(msg)
540 end
541
542 function edit_selected()
543     local n = selected_rally_number()
544     if not n then set_status("Pick a rally in the Recent list first, then Edit
selected."); return end
545     local idx = index_of_rally(n)
546     if not idx then set_status("Rally #" .. tostring(n) .. " not found (try Refresh).");
return end
547     local r = rows[idx]
548     mode = "edit"; edit_index = idx
549     w_start:set_text(string.format("%.3f", r.s))
550     w_end:set_text(string.format("%.3f", r.e))
551     rebuild_reason_selected(r.reason)
552     rebuild_sport_selected(r.sport)
553     refresh_buttons()
554     set_status(string.format(
555         "Editing rally #%d. Adjust Start/End/reason, then Save changes. (Undo last
cancels.)", r.n))
556 end
557
558 function delete_selected()

```

```

559     local n = selected_rally_number()
560     if not n then set_status("Pick a rally in the Recent list first, then Delete
selected."); return end
561     local idx = index_of_rally(n)
562     if not idx then set_status("Rally #" .. tostring(n) .. " not found (try Refresh).");
return end
563     table.remove(rows, idx)
564     local ok, err = save_all()
565     if not ok then set_status("WRITE FAILED: " .. tostring(err)); return end
566     reset_form()
567     refresh_list()
568     set_status(string.format("Deleted rally #%d. %d remaining.", n, #rows))
569 end
570
571 function undo_last()
572     if mode == "edit" then
573         reset_form(); refresh_list(); set_status("Edit cancelled.")
574         return
575     end
576     local s = get_field_num(w_start)
577     local e = get_field_num(w_end)
578     if s or e then
579         reset_form()
580         set_status("Cleared the in-progress START/END (nothing was written).")
581         return
582     end
583     if #rows == 0 then set_status("Nothing to undo."); return end
584     local last = rows[#rows]
585     rows[#rows] = nil
586     local ok, err = save_all()
587     if not ok then rows[#rows + 1] = last; set_status("WRITE FAILED: " .. tostring(err));
return end
588     refresh_list()
589     refresh_buttons()
590     set_status(string.format("Removed last rally #%d. %d remaining.", last.n, #rows))
591 end
592
593 function refresh_now()
594     sync_to_current_video()
595 end
596
597 function show_help()
598     -- Toggle a DEDICATED help panel by adding/removing a widget. In-place set_text on
599     -- the shared status widget did not repaint reliably in VLC (the panel got "stuck");
600     -- add_html + del_widget forces a real layout change that VLC renders every time.
601     if w_help then
602         if d then d:del_widget(w_help) end
603         w_help = nil
604         if w_help_btn then w_help_btn:set_text("Help") end
605     else
606         if d then w_help = d:add_html(HELP_HTML, 1, 14, 4, 8) end
607         if w_help_btn then w_help_btn:set_text("Hide help") end
608     end
609     if d then d:update() end
610 end
611
612 -----
613 -- Dialog construction
614 -----
615 local function create_dialog()
616     d = vlc.dialog("Rally Annotator")
617
618     d:add_label("Sport:", 1, 1, 1, 1)
619     w_sport = d:add_dropdown(2, 1, 2, 1)
620     for i, v in ipairs(SPORTS) do w_sport:add_value(v, i) end    -- badminton first =>

```

```

default
621     w_help_btn = d:add_button("Help", show_help, 4, 1, 1, 1)
622
623     d:add_label("Ending reason (required):", 1, 2, 1, 1)
624     w_reason = d:add_dropdown(2, 2, 1, 1)
625     d:add_label("Next rally #:", 3, 2, 1, 1)
626     w_next = d:add_text_input("", 4, 2, 1, 1)
627
628     d:add_label("Start (s):", 1, 3, 1, 1)
629     w_start = d:add_text_input("", 2, 3, 1, 1)
630     d:add_label("End (s):", 3, 3, 1, 1)
631     w_end = d:add_text_input("", 4, 3, 1, 1)
632
633     d:add_button("Mark START", mark_start, 1, 4, 1, 1)
634     d:add_button("Mark END", mark_end, 2, 4, 1, 1)
635     w_save = d:add_button("Save Rally", save_rally, 3, 4, 2, 1)
636
637     w_status = d:add_html("", 1, 5, 4, 2) -- rich-text so multi-line (<br>) renders;
also hosts Help
638
639     d:add_label("Recent rallies (select one, then Edit/Delete):", 1, 7, 4, 1)
640     w_list = d:add_list(1, 8, 4, 4)
641
642     d:add_button("Edit selected", edit_selected, 1, 12, 1, 1)
643     d:add_button("Delete selected", delete_selected, 2, 12, 1, 1)
644     w_undo = d:add_button("Undo last", undo_last, 3, 12, 1, 1)
645     d:add_button("Refresh", refresh_now, 4, 12, 1, 1)
646
647     rebuild_reason_placeholder()
648     refresh_next_field()
649     refresh_list()
650     refresh_buttons()
651     d:show()
652     if #rows > 0 then
653         set_status(string.format(
654             "Resumed this video: %d existing rallies loaded (next is #%d). Click Help for the
guide.",
655             #rows, next_rally_number()))
656     else
657         set_status("Ready. Pick the Sport, then mark rallies. Click Help for usage + the
ending-reason guide.")
658     end
659 end
660
661 -----
662 -- Lifecycle
663 -----
664 function activate()
665     -- Build the id lookups here (full Lua environment), not at top level (see note
666     -- where REASON_ID/SPORT_ID are declared -- the descriptor scan sandbox lacks ipairs).
667     for i, v in ipairs(REASONS) do REASON_ID[v] = i end
668     for i, v in ipairs(SPORTS) do SPORT_ID[v] = i end
669     mode = "new"
670     edit_index = nil
671     w_help = nil
672     out_path = resolve_out_path()
673     load_rows() -- continue an existing CSV (numbering + recent list)
674     create_dialog()
675 end
676
677 function deactivate()
678     if d then d:delete(); d = nil end
679     w_help = nil
680 end
681

```

```
682     function close()
683         deactivate()
684         vlc.deactivate()
685     end
686
```

4. assistant (2026-06-08T11:34:45.384Z)

I've read the file in full. Now let me analyze the resume feature logic carefully, tracing through the key flows and edge cases.

Let me trace the critical paths:

```
**EDIT mode (line 518-523):** Uses `rows[edit_index]` and keeps `r.n` unchanged ? correctly does NOT use `planned_next_number()`. Good.
```

```
**Save button label in edit mode (line 383-384):** Uses `rows[edit_index].n` ? correct.
```

Now let me examine the interaction between `delete\_selected`/`undo\_last` and the `w\_next` field, plus the auto-advance/gap insertion logic, which are the areas most likely to harbor defects.

Let me trace `delete\_selected` (line 558-568) and `undo\_last` (line 571-591): both call `reset\_form()` / `refresh\_list()` but neither calls `refresh\_next\_field()`. So `w\_next` retains its prior value. Let me verify what `reset\_form` does ? it does NOT touch `w\_next`. Confirmed.

Let me trace the gap-insertion auto-advance carefully. Suppose rows are #1, #5. User sets `w\_next=2`, saves ? row #2 appended, `w\_next` set to 3. Saves again ? `planned\_next\_number()` reads 3, `index\_of\_rally(3)` is nil, appends #3, sets `w\_next=4`. Continues 4, then 5 ? `index\_of\_rally(5)` exists ? refused. So auto-advance can walk into an existing number and get blocked. Let me confirm the refusal leaves state consistent.

Let me check one more subtle thing about `refresh\_buttons` being called after marks but the `w\_next` field not being re-read live, and whether the label can go stale vs. what save writes.

5. user (2026-06-08T11:34:46.825Z)

```
140:local w_next          -- "Next rally #" text input (lets you resume numbering anywhere)
304:local function planned_next_number()
305:  local v = get_field_num(w_next)
311:local function refresh_next_field()
312:  if w_next then w_next:set_text(tostring(next_rally_number())) end
381:local function refresh_buttons()
389:  w_save:set_text(string.format("Save Rally (%d)", planned_next_number()))
441:  refresh_buttons()
452:  refresh_list(); refresh_buttons()
458:  refresh_list(); refresh_buttons()
465:  refresh_next_field()
466:  refresh_list(); refresh_buttons()
473:  refresh_next_field()
475:  refresh_buttons()
487:  refresh_buttons()
495:  refresh_buttons()
525:  local n = planned_next_number()
533:  if w_next then w_next:set_text(tostring(n + 1)) end  -- continue numbering from here
553:  refresh_buttons()
589:  refresh_buttons()
626:  w_next = d:add_text_input("", 4, 2, 1, 1)
648:  refresh_next_field()
650:  refresh_buttons()
```

6. assistant (2026-06-08T11:35:02.669Z)

Now I have a complete picture. Let me verify a few specific defects by re-reading the precise logic.

****Stale `w\_next` button label:**** The `w\_save` label is only refreshed via `refresh\_buttons()`, which is NOT called when the user edits the `w\_next` text field directly (VLC text inputs don't fire callbacks on every keystroke, and there's no callback wired to `w\_next` at all). So if the user types a new "Next rally #", the Save button label still shows the OLD planned number until some other action triggers `refresh\_buttons`. But `save\_rally` re-reads `planned\_next\_number()` fresh, so it writes the correct (typed) value. This is a label/actual mismatch, but a minor one. Let me verify there's no callback on `w\_next`.

Confirmed at line 626: `w\_next = d:add\_text\_input("", 4, 2, 1, 1)` ? no callback. The label can be stale relative to a freshly-typed value, but `save\_rally` writes correctly.

Now let me confirm the most important defect: the ****delete-then-save surprising-number**** interaction, and the ****w_next not reset after delete/undo****.

Scenario A (undo): rows #1,#2,#3. `w\_next` shows "4". User clicks "Undo last (#3)" ? row #3 removed, `w\_next` STILL shows "4" (undo never calls `refresh\_next\_field`). Now `next\_rally\_number()` would be 3, but the field says 4. User marks a new rally and saves ? `planned\_next\_number()` reads 4 ? writes #4, creating a ****gap**** (#1,#2,#4). This is surprising: the user undid #3 expecting the next save to reproduce #3.

Scenario B (delete): rows #1,#2,#3, `w\_next`="4". User deletes #2 ? rows #1,#3. `delete\_selected` calls `reset\_form()` (doesn't touch `w\_next`) ? field still "4". Next save ? reads 4 ? #4. Fine here, but if `w\_next` had been left at "2" from a prior manual set, deleting could produce odd results. The core issue is delete/undo never re-sync `w\_next`.

Let me confirm `reset\_form` (435-443) doesn't reset `w\_next` ? confirmed, it only touches start/end/reason.

I have enough to report concrete defects. Let me also double check the collision-check-bypass and the fractional/negative handling once more.

`planned\_next\_number` (304-308): `if v and v >= 1 then return math.floor(v)`. So `w\_next=2.9` ? floor ? 2. `w\_next=0`, negative, blank, or non-numeric ? falls to `next\_rally\_number()`. The collision check at 526 uses this floored value and `index\_of\_rally`, which compares against stored integer `n`. No bypass: floored value is an integer, stored `n` are integers. Good.

One more: fractional like `2.9` shows in the field as "2.9" but writes #2 ? the Save label (`planned\_next\_number`) also floors to 2, so label and write agree there. Consistent.

Now I'll compile the concrete defects.

7. user (2026-06-08T11:35:21.958Z)

Structured output provided successfully

8. assistant (2026-06-08T11:35:35.447Z)

I reviewed the logic and edge cases of the "Next rally #" resume feature in `C:\Users\avidu\Projects\rally-annotator\vlc\rally\_annotator.lua`.

****Things that are correct**** (so you don't worry about them):

- EDIT mode (lines 518-523) keeps the existing `r.n` and does NOT use `planned\_next\_number()`; the Save label in edit mode (lines 383-384) also uses `rows[edit\_index].n`. Correct.
- Blank/0/negative/non-numeric `w\_next` all fall back to `next\_rally\_number()` (line 305-307); fractional values are floored. No collision-check bypass ? the floored integer is compared against integer `r.n` (line 526).
- Refused duplicate leaves state consistent: the check at line 526 returns before any `rows` mutation or write, and `w\_next` is untouched.

****Concrete defects found:****

1. ****Undo doesn't re-sync `w\_next` (bug, high confidence)**** ? `undo\_last` (lines 571-591) removes the last row but never calls `refresh\_next\_field()`. With rows #1,#2,#3 and `w\_next`="4", undoing #3 leaves `w\_next`="4", so the next new save writes #4 (gap #1,#2,#4) instead of reproducing #3. This is the surprising-next-number interaction you flagged.

2. ****Delete doesn't re-sync `w\_next` (bug, high confidence)**** ? `delete\_selected` (lines 558-568) calls `reset\_form()` (which deliberately leaves `w\_next` alone, lines 435-443) but never `refresh\_next\_field()`. Deleting the top-numbered rally leaves `w\_next` pointing past the freed number. This is inconsistent with `sync\_to\_current\_video`, which *does* call `refresh\_next\_field()` (lines 464-465, 473).

3. ****Save button label can go stale vs. what Save writes (risk, medium confidence)**** ? `w\_next` (line 626) has no on-change callback, and the `Save Rally (#N)` label is only recomputed inside `refresh\_buttons()`, which isn't triggered by editing the Next rally # field. So after the user types a new number, the button may still show the old `#N` while `save\_rally` correctly writes the typed value (it re-reads `planned\_next\_number()` at line 525). Display mismatch only ? the write is correct.

4. ****Auto-advance can step into an occupied number (nit, medium confidence)**** ? with a gap (e.g. rows #1,#5), auto-advance walks 2?3?4?5 and only discovers the collision at save time, refusing with "Rally #5 already exists". Safe (no overwrite, state consistent) but mildly surprising; it could skip occupied slots instead.

The fix for #1 and #2 is the same: call `refresh\_next\_field()` after a successful `save\_all()` in both `undo\_last` and `delete\_selected`, matching the Refresh path's behavior.